SQL Retail Analytics Project

1. Create Tables

```
CREATE TABLE customers (
customer_id NUMBER,
customer_name VARCHAR2(50)
);

CREATE TABLE stores (
store_id NUMBER,
store_name VARCHAR2(50)
);

CREATE TABLE products (
product_id NUMBER,
category VARCHAR2(50)
);

CREATE TABLE sales (
sale_id NUMBER,
customer_id NUMBER,
store_id NUMBER,
product_id NUMBER,
sale_date DATE,
amount NUMBER,
quantity NUMBER,
discount NUMBER
);
```

2. Insert Data (Auto Generate 1000 Rows)

```
-- Customers
INSERT INTO customers
SELECT LEVEL, 'Cust_'||LEVEL FROM dual CONNECT BY LEVEL<=50;

-- Stores
INSERT INTO stores VALUES (1,'CMR');
INSERT INTO stores VALUES (2,'SR');
```

```
INSERT INTO stores VALUES (3,'Lucky');

-- Products
INSERT INTO products VALUES (1,'Electronics');
INSERT INTO products VALUES (2,'Fashion');
INSERT INTO products VALUES (3,'Grocery');
INSERT INTO products VALUES (4,'Home');

-- Sales (1000 rows)
INSERT INTO sales
SELECT
LEVEL,
MOD(LEVEL,50)+1,
MOD(LEVEL,3)+1,
MOD(LEVEL,4)+1,
DATE '2025-01-01' + MOD(LEVEL,90),
ROUND(DBMS_RANDOM.VALUE(100,10000)),
MOD(LEVEL,5)+1,
CASE WHEN MOD(LEVEL,4)=0 THEN NULL ELSE ROUND(DBMS_RANDOM.VALUE(10,500))
END
FROM dual
CONNECT BY LEVEL <= 1000;
```

NOTE:Due to space constraints only 5 records were pasted as output

1. Data Understanding

Manager Requirement:
Give me overall business performance summary for the last quarter.

Questions:

1. Find total number of transactions.
   select count(*) as Total_No_Transactions from sales5;

output:

	TOTAL_NO_TRANSACTIONS
1	1000

2. Find total revenue generated.
   select sum(amount) as revenue from sales5
   output:

REVENUE
5055520

3. Find total unique customers.

```
select count(distinct(customer_id)) as Unique_customer from customers5
```

output:

	UNIQUE_CUSTOMER
1	50

4. Find average order value.

```
select avg(amount) as Avg_order_value from sales5
```

output:

	AVG_ORDER_VALUE
1	5055.52

5. Find total quantity sold

```
select sum(quantity) as Total_quantity_sold from sales5
```

output:

	TOTAL_QUANTITY_
1	3000

6. Find minimum and maximum sale amount.

```
select min(amount) as Min_amt,max(amount) as Max_amt from sales5
```

output:

	MIN_AMT	MAX_AMT
1	101	9978

7. Find average discount given

```
select round(avg(discount),2) as Avg_discount from sales5
```

output:

	AVG_DISCOUNT
1	261.32

8. Find total sales per store.

```
select s1.store_id,s1.store_name,sum(s2.amount) as Total_sales from stores5 s1
left join sales5 s2
on s1.store_id=s1.store_id
group by s1.store_id,s1.store_name
```

output:

	STORE_ID	STORE_NAME	TOTAL_SALES
1	1	CMR	5055520
2	2	SR	5055520
3	3	Lucky	5055520

9. Find total sales per category.

```
select p.product_id ,p.category,sum(s.amount)as Total_sales from products5 p
left join sales5 s
on p.product_id=s.product_id
group by p.product_id ,p.category
```

output:

	PRODUCT_ID	CATEGORY	TOTAL_SALES
1	1	Electronics	1254138
2	2	Fashion	1296327
3	3	Grocery	1202791
4	4	Home	1302264

10. Find number of transactions per day.

```
select sale_date,count(sale_id) as No_of_transactions from sales5
group by sale_date
```

output:

	SALE_DATE	NO_OF_TRANSACTI
1	2/18/2025, 12:00:0(	11
2	2/19/2025, 12:00:0(	11
3	2/20/2025, 12:00:0(	11
4	2/21/2025, 12:00:0(	11
5	2/22/2025, 12:00:0(	11

+45 rows

---

2. Data Cleaning

Manager Requirement:
Data looks inconsistent, fix NULL issues before analysis.

Questions:

11. Replace NULL discount with 0.

```
select sale_id,nvl(discount,0) as discount from sales5
```

output:

	SALE_ID	DISCOUNT
1	408	0
2	409	80
3	410	126
4	411	352
5	412	0

+995 rows

12. Find number of NULL discounts.

```
select count(*) from sales5
  where discount is null
```

output:

	COUNT(*)
1	250

13. Calculate net revenue (amount - discount).

```
select sum(amount-nvl(discount,0)) as revenue from sales5
```

output:

	REVENUE
1	4859528

14. Replace NULL quantity with 1.

```
select nvl(quantity,1) from sales5
```

output:

	NVL(QUANTITY,1)
1	4
2	5
3	1
4	2
5	3

+995 rows

15. Identify rows where amount is NULL.

```
select sale_id,customer_id from sales5 where amount is null
```

output:

SALE_ID	CUSTOMER_ID
No items to display.	

*In given data there is no null amount

16. Use COALESCE to handle multiple NULL columns.
    select sale_id,COALESCE (amount,quantity,discount,0) as value from sales5

output:

	SALE_ID	VALUE
1	408	8852
2	409	2366
3	410	3858
4	411	8599
5	412	2221

17. Create a cleaned column for revenue.
    select sale_id,nvl(amount,0)-nvl(discount,0) as cleaned_revenue from sales5

output:

	SALE_ID	CLEANED_REVENUE
1	408	8852
2	409	2286
3	410	3732
4	411	8247
5	412	2221

18. Check percentage of missing data.
    SELECT
    ROUND(AVG(CASE WHEN amount IS NULL THEN 1 ELSE 0 END) * 100, 2) AS amount_missing_pct,
    ROUND(AVG(CASE WHEN discount IS NULL THEN 1 ELSE 0 END) * 100, 2) AS discount_missing_pct,
    ROUND(AVG(CASE WHEN quantity IS NULL THEN 1 ELSE 0 END) * 100, 2) AS quantity_missing_pct
    FROM sales5;

    Output:

	AMOUNT_MISSING_	DISCOUNT_MISSIN(	QUANTITY_MISSIN(
1	0	25	0

19. Flag rows with missing values.

```
select sale_id,
CASE
   when amount is null or discount is null then 'missing'
   else 'complete'
   end as flag
   from sales5
```

output:

	SALE_ID	FLAG
1	408	missing
2	409	complete
3	410	complete
4	411	complete
5	412	missing

---

3. Filtering (WHERE)

Manager Requirement:
I want specific insights based on conditions.

Questions:

20. Find sales in last 30 days.

```
select * from sales5
 where sale_date>=sysdate-30
0
```

21. Find sales above 5000.

```
select * from sales5 where amount>5000
```

output:

	SALE_ID	CUSTOMER_ID	STORE_ID	PRODUCT_ID	SALE_DATE	AMOUNT	QUANTITY	DISCOUNT
1	408	9	1	1	2/18/2025, 12:00:0(	8852	4	(null)
2	411	12	1	4	2/21/2025, 12:00:0(	8599	2	352
3	413	14	3	2	2/23/2025, 12:00:0(	8529	4	395
4	414	15	1	3	2/24/2025, 12:00:0(	7334	5	65
5	416	17	3	1	2/26/2025, 12:00:0(	5897	2	(null)

22. Find sales between 1000 and 5000.

```
select * from sales5 where amount>1000 and amount<5000
```

output:

	SALE_ID	CUSTOMER_ID	STORE_ID	PRODUCT_ID	SALE_DATE	AMOUNT	QUANTITY	DISCOUNT
1	409	10	2	2	2/19/2025, 12:00:0(	2366	5	80
2	410	11	3	3	2/20/2025, 12:00:0(	3858	1	126
3	412	13	2	1	2/22/2025, 12:00:0(	2221	3	(null)
4	415	16	2	4	2/25/2025, 12:00:0(	3776	1	476
5	418	19	2	3	2/28/2025, 12:00:0(	2060	4	307

23. Find sales from CMR store.

```
select s.store_id,store_name,product_id,quantity,amount,nvl(discount,0) from sales5 s
left join stores5 ss
on s.store_id=ss.store_id
where store_name='CMR'
```

output:

	STORE_ID	STORE_NAME	PRODUCT_ID	QUANTITY	AMOUNT	NVL(DISCOUNT,0)
1	1	CMR	1	4	8852	0
2	1	CMR	4	2	8599	352
3	1	CMR	3	5	7334	65
4	1	CMR	2	3	6074	109
5	1	CMR	1	1	1036	0

24. Find Electronics category sales.

```
select p.product_id,p.category,sum(amount) as Amount from products5 p
join sales5 s
on p.product_id=s.product_id
where category='Electronics'
group by  p.product_id,p.category
```

output:

	PRODUCT_ID	CATEGORY	AMOUNT
1	1	Electronics	1254138

25. Find sales excluding Grocery category.

```
SELECT *
FROM sales5 s
LEFT JOIN products5 p
ON s.product_id = p.product_id
WHERE p.category IS NULL OR p.category <> 'Grocery';
```

26. Find customers from specific store.

```
SELECT DISTINCT c.customer_id,
                c.customer_name,
                s.store_id,
                st.store_name
    FROM sales5 s
    JOIN customers5 c
      ON s.customer_id = c.customer_id
    JOIN stores5 st
      ON s.store_id = st.store_id
```

```
WHERE st.store_name = 'CMR';
```

Output:

	CUSTOMER_ID	CUSTOMER_NAME	STORE_ID	STORE_NAME
1	9	Cust_9	1	CMR
2	12	Cust_12	1	CMR
3	15	Cust_15	1	CMR
4	18	Cust_18	1	CMR
5	21	Cust_21	1	CMR

27. Find high discount transactions.

```
SELECT sale_id,
       customer_id,
       store_id,
       product_id,
       amount,
       NVL(discount,0) AS discount,
       sale_date
FROM sales5
WHERE NVL(discount,0) > 300;
```

Output:

	SALE_ID	CUSTOMER_ID	STORE_ID	PRODUCT_ID	AMOUNT	DISCOUNT	SALE_DATE
1	411	12	1	4	8599	352	2/21/2025, 12:00:0
2	413	14	3	2	8529	395	2/23/2025, 12:00:0
3	415	16	2	4	3776	476	2/25/2025, 12:00:0
4	418	19	2	3	2060	307	2/28/2025, 12:00:0
5	422	23	3	3	8008	411	3/4/2025, 12:00:00

28. Find low quantity sales.

```
SELECT * FROM sales5
WHERE quantity < 2;
```

Output:

	SALE_ID	CUSTOMER_ID	STORE_ID	PRODUCT_ID	SALE_DATE	AMOUNT	QUANTITY	DISCOUNT
1	410	11	3	3	2/20/2025, 12:00:0	3858	1	126
2	415	16	2	4	2/25/2025, 12:00:0	3776	1	476
3	420	21	1	1	3/2/2025, 12:00:00	1036	1	*(null)*
4	425	26	3	2	3/7/2025, 12:00:00	5864	1	288
5	430	31	2	3	3/12/2025, 12:00:0	3316	1	409

29. Find recent transactions.

```
SELECT *
FROM sales5
WHERE sale_date >= (SELECT MAX(sale_date) - 7 FROM sales5);
```

Output:

	Show statement	CUSTOMER_ID	STORE_ID	PRODUCT_ID	SALE_DATE	AMOUNT	QUANTITY	DISCOUNT
1	442	43	2	3	3/24/2025, 12:00:0(	3767	3	226
2	443	44	3	4	3/25/2025, 12:00:0(	2723	4	378
3	444	45	1	1	3/26/2025, 12:00:0(	821	5	(null)
4	445	46	2	2	3/27/2025, 12:00:0(	8114	1	109
5	446	47	3	3	3/28/2025, 12:00:0(	6644	2	355

---

4. Aggregation (GROUP BY)

Manager Requirement:
Break down performance across business segments.

Questions:

30. Revenue per store.

```
select s1.store_id,s1.store_name,sum(amount) as revenue from stores5 s1
left join sales5 s2
on s1.store_id=s2.store_id
group by  s1.store_id,s1.store_name
```

output:

STORE_ID	STORE_NAME	REVENUE
1	CMR	1651758
2	SR	1718445
3	Lucky	1685317

31. Revenue per category.

```
select p.product_id,p.category,sum(nvl(amount,0)) as revenue from products5 p
left join sales5 s
on p.product_id=s.product_id
group by   p.product_id,p.category
```

output:

PRODUCT_ID	CATEGORY	REVENUE
1	Electronics	1254138
2	Fashion	1296327
3	Grocery	1202791
4	Home	1302264

32. Revenue per customer.

```
select c.customer_id,sum(amount) as revenue
from customers5 c
left join sales5 s
on c.customer_id=s.customer_id
group by c.customer_id
```

output:

	CUSTOMER_ID	REVENUE
1	9	93677
2	10	115763
3	11	94592
4	12	106733
5	13	103449

33. Monthly revenue

```
SELECT TO_CHAR(sale_date, 'YYYY-MM') AS month,
NVL(SUM(amount),0) AS revenue
 FROM sales5
  GROUP BY TO_CHAR(sale_date, 'YYYY-MM')
  ORDER BY month;
```

Output:

	MONTH	REVENUE
1	2025-01	1813216
2	2025-02	1565707
3	2025-03	1676597

34. Daily revenue.

```
select to_char(sale_date,'dd-mm-yyyy') as daily,
nvl(sum(amount),0) as revenue
```

```
from sales5
group by to_char(sale_date,'dd-mm-yyyy')
order by daily
```

output:

	DAILY	REVENUE
1	01-01-2025	49096
2	01-02-2025	71930
3	01-03-2025	57015
4	02-01-2025	63773
5	02-02-2025	52699

35. Average sales per store.

```
select to_char(sale_date,'dd-mm-yyyy') as daily,
round(nvl(avg(amount),0),2) as avg_sales
from sales5
group by to_char(sale_date,'dd-mm-yyyy')
order by daily
```

output:

	DAILY	AVG_SALES
1	01-01-2025	4463.27
2	01-02-2025	6539.09
3	01-03-2025	5183.18
4	02-01-2025	5314.42
5	02-02-2025	4790.82

36. Total quantity per category.

```
select p.category,NVL(SUM(s.quantity),0) as Total_Quantity from products p
left join sales s
 on p.product_id=s.product_id
group by p.category
```

output:

CATEGORY	TOTAL_QUANTITY
Fashion	5
Electronics	4
Accessories	5

37. Total discount per store.

```
SELECT st.store_name,
    NVL(SUM(s.discount),0) AS total_discount
FROM stores5 st
```

```
LEFT JOIN sales5 s
ON st.store_id = s.store_id
GROUP BY st.store_name
ORDER BY st.store_name;
```

Output:

	STORE_NAME	TOTAL_DISCOUNT
1	CMR	64069
2	Lucky	67256
3	SR	64667

38. Customer-wise total spending.

```
SELECT c.customer_id,
    c.customer_name,
    NVL(SUM(s.amount),0) AS total_spent
FROM customers5 c
LEFT JOIN sales5 s
ON c.customer_id = s.customer_id
GROUP BY c.customer_id, c.customer_name
ORDER BY total_spent DESC;
```

Output:

CUSTOMER_ID	CUSTOMER_NAME	TOTAL_SPENT
22	Cust_22	127374
36	Cust_36	125249
18	Cust_18	125166
46	Cust_46	125139
47	Cust_47	123382

39. Store-wise transaction count.

```
SELECT st.store_name,
    COUNT(s.sale_id) AS transaction_count
FROM stores5 st
LEFT JOIN sales5 s
ON st.store_id = s.store_id
GROUP BY st.store_name
ORDER BY transaction_count DESC;
```

Output:

	STORE_NAME	TRANSACTION_COU
1	SR	334
2	Lucky	333
3	CMR	333

---

5. HAVING

Manager Requirement:
Find top performing segments.

Questions:

40. Customers with revenue > 20000.

```
select customer_id,nvl(sum(amount),0) as revenue   from sales5
group by customer_id
having  nvl(sum(amount),0) >20000
```

output:

	CUSTOMER_ID	REVENUE
1	9	93677
2	10	115763
3	11	94592
4	12	106733
5	13	103449

41. Stores with revenue > 100000.

```
select s.store_id,s.store_name,nvl(sum(amount),0) as Store_revenue from stores5 s
left join sales5 ss
on s.store_id=ss.store_id
group by s.store_id,s.store_name
having nvl(sum(amount),0)>100000
```

output;

	STORE_ID	STORE_NAME	STORE_REVENUE
1	1	CMR	1651758
2	2	SR	1718445
3	3	Lucky	1685317

42. Categories with avg sales > 3000.

```
select p.category,round(nvl(avg(amount),0),2)  as avg_sales from products5 p
```

```
left join sales5 s
on p.product_id=s.product_id
group by p.category
having nvl(avg(amount),0)>3000
```

output:

	CATEGORY	AVG_SALES
1	Electronics	5016.55
2	Fashion	5185.31
3	Grocery	4811.16
4	Home	5209.06

43. Customers with more than 5 transactions.

```
SELECT customer_id,
    COUNT(sale_id) AS transaction_count
FROM sales5
GROUP BY customer_id
HAVING COUNT(sale_id) > 5;
```

Output:

	CUSTOMER_ID	TRANSACTION_COU
1	9	20
2	10	20
3	11	20
4	12	20
5	13	20

44. Stores with high discount usage.

```
select s1.store_id,s1.store_name,nvl(sum(discount),0) as Total_discount from stores5 s1
left join sales5 s2
on s1.store_id=s2.store_id
group by s1.store_id,s1.store_name
having nvl(sum(discount),0) >65000
```

output:

	STORE_ID	STORE_NAME	TOTAL_DISCOUNT
1	3	Lucky	67256

45. Categories with low performance.

```
SELECT p.category,
    NVL(SUM(s.amount),0) AS total_revenue
FROM products5 p
LEFT JOIN sales5 s
ON p.product_id = s.product_id
GROUP BY p.category
HAVING NVL(SUM(s.amount),0) < 90000;
```

46. Customers with high avg order value.

```
SELECT customer_id,
    ROUND(NVL(AVG(amount),0),2) AS avg_order_value
FROM sales5
GROUP BY customer_id
HAVING NVL(AVG(amount),0) > 5000;
```

Output:

	CUSTOMER_ID	AVG_ORDER_VALUI
1	10	5788.15
2	12	5336.65
3	13	5172.45
4	15	5974.4
5	16	5023.4

47. Find top 5 customers using HAVING.

```
SELECT customer_id,
    SUM(amount) AS total_revenue
FROM sales5
GROUP BY customer_id
ORDER BY total_revenue DESC
FETCH FIRST 5 ROWS ONLY;
```

Output:

	CUSTOMER_ID	TOTAL_REVENUE
1	22	127374
2	36	125249
3	18	125166
4	46	125139
5	47	123382

48. Stores with max transactions.

```
SELECT s.store_id,
       st.store_name,
       COUNT(s.sale_id) AS transaction_count
FROM sales5 s
JOIN stores5 st
ON s.store_id = st.store_id
GROUP BY s.store_id, st.store_name
HAVING COUNT(s.sale_id) = (
  SELECT MAX(cnt)
  FROM (
    SELECT COUNT(sale_id) AS cnt
    FROM sales5
    GROUP BY store_id
  )
);
```

Output:

STORE_ID	STORE_NAME	TRANSACTION_COU
2	SR	334

49. Categories with high growth.

```
SELECT p.category,
       SUM(CASE WHEN TO_CHAR(s.sale_date,'MM') = '02' THEN s.amount ELSE 0 END)
AS feb_sales,
       SUM(CASE WHEN TO_CHAR(s.sale_date,'MM') = '01' THEN s.amount ELSE 0 END)
AS jan_sales
FROM sales5 s
JOIN products5 p
ON s.product_id = p.product_id
GROUP BY p.category
HAVING SUM(CASE WHEN TO_CHAR(s.sale_date,'MM') = '02' THEN s.amount ELSE 0
END) >
       SUM(CASE WHEN TO_CHAR(s.sale_date,'MM') = '01' THEN s.amount ELSE 0 END);
```

6. CASE WHEN

Manager Requirement:
Segment customers for business decisions.

Questions:

50. Classify customers as High/Medium/Low spenders.

```
SELECT c.customer_id,
        c.customer_name,
        CASE
          WHEN NVL(SUM(s.amount),0) < 100000 THEN 'Low spender'
          WHEN NVL(SUM(s.amount),0) < 500000 THEN 'Medium spender'
          ELSE 'High spender'
        END AS shopping_category
    FROM customers5 c
    LEFT JOIN sales5 s
    ON c.customer_id = s.customer_id
    GROUP BY c.customer_id, c.customer_name;
```

Output:

CUSTOMER_ID	CUSTOMER_NAME	SHOPPING_CATEGC
9	Cust_9	Low spender
10	Cust_10	Medium spender
11	Cust_11	Low spender
12	Cust_12	Medium spender
13	Cust_13	Medium spender

51. Classify transactions as Big/Small.

```
SELECT sale_id,
        CASE
      WHEN amount > 5000 THEN 'Big'
          ELSE 'Small'
        END AS transaction_category
    FROM sales5;
```

Output:

	SALE_ID	TRANSACTION_CAT
1	408	Big
2	409	Small
3	410	Small
4	411	Big
5	412	Small

52. Create discount flag.

```
SELECT sale_id,
```

```
    CASE
      WHEN discount IS NULL OR discount = 0 THEN 'No Discount'
      ELSE 'Discount Applied'
    END AS discount_flag
FROM sales5;
```

Output:

	SALE_ID	DISCOUNT_FLAG
1	408	No Discount
2	409	Discount Applied
3	410	Discount Applied
4	411	Discount Applied
5	412	No Discount

53. Segment stores based on revenue.

```
SELECT s.store_id,
    st.store_name,
    SUM(s.amount) AS revenue,
    CASE
      WHEN SUM(s.amount) > 2000000 THEN 'High'
      WHEN SUM(s.amount) > 1000000 THEN 'Medium'
      ELSE 'Low'
    END AS store_segment
FROM sales5 s
JOIN stores5 st
ON s.store_id = st.store_id
GROUP BY s.store_id, st.store_name
```

Output:

STORE_ID	STORE_NAME	REVENUE	STORE_SEGMENT
1	CMR	1651758	Medium
2	SR	1718445	Medium
3	Lucky	1685317	Medium

54. Categorize customers based on frequency.

```
SELECT customer_id,
    COUNT(sale_id) AS transactions,
    CASE
      WHEN COUNT(sale_id) > 20 THEN 'Frequent'
      WHEN COUNT(sale_id) > 10 THEN 'Regular'
      ELSE 'Occasional'
    END AS customer_type
FROM sales5
GROUP BY customer_id;
```

Output:

	CUSTOMER_ID	TRANSACTIONS	CUSTOMER_TYPE
1	9	20	Regular
2	10	20	Regular
3	11	20	Regular
4	12	20	Regular
5	13	20	Regular

55. Create sales buckets.

```
SELECT sale_id,
    CASE
      WHEN amount < 1000 THEN 'Low'
      WHEN amount < 5000 THEN 'Medium'
      ELSE 'High'
    END AS sales_bucket
FROM sales5;
```

Output:

	SALE_ID	SALES_BUCKET
1	408	High
2	409	Medium
3	410	Medium
4	411	High
5	412	Medium

56. Identify premium customers.

```
SELECT customer_id,
    SUM(amount) AS total_spent
FROM sales5
GROUP BY customer_id
HAVING SUM(amount) > 50000;
```

Output:

	CUSTOMER_ID	TOTAL_SPENT
1	9	93677
2	10	115763
3	11	94592
4	12	106733
5	13	103449

57. Mark high-value transactions.

```
SELECT sale_id,
    CASE
      WHEN amount > 7000 THEN 'High Value'
      ELSE 'Normal'
    END AS transaction_type
```

```
FROM sales5;
```

Output:

	SALE_ID	TRANSACTION_TYF
1	408	High Value
2	409	Normal
3	410	Normal
4	411	High Value
5	412	Normal

58. Classify categories based on performance.

```
SELECT p.category,
        SUM(s.amount) AS revenue,
        CASE
          WHEN SUM(s.amount) > 100000 THEN 'High'
          WHEN SUM(s.amount) > 50000 THEN 'Medium'
          ELSE 'Low'
        END AS performance
    FROM sales5 s
    JOIN products5 p
    ON s.product_id = p.product_id
    GROUP BY p.category;
```

Output:

	CATEGORY	REVENUE	PERFORMANCE
1	Electronics	1254138	High
2	Fashion	1296327	High
3	Grocery	1202791	High
4	Home	1302264	High

59. Create custom labels for reporting.

```
SELECT sale_id,
        customer_id,
        amount,
        CASE
          WHEN amount > 5000 AND discount IS NOT NULL THEN 'High Value + Discount'
          WHEN amount > 5000 THEN 'High Value'
          WHEN discount IS NOT NULL THEN 'Discounted Sale'
          ELSE 'Regular Sale'
        END AS label
    FROM sales5;
```

Output:

	SALE_ID	CUSTOMER_ID	AMOUNT	LABEL
1	408	9	8852	High Value
2	409	10	2366	Discounted Sale
3	410	11	3858	Discounted Sale
4	411	12	8599	High Value + Discour
5	412	13	2221	Regular Sale

---

7. JOINS (Multiple Tables)

Manager Requirement:
Combine customer, product, and store data for insights.

Questions:

60. Join sales with customers table.

```
select  s.sale_id,
        s.sale_date,
        s.amount,
        c.customer_id,
        c.customer_name from  sales5 s
left join customers5 c
on s.customer_id=c.customer_id
```

output:

	SALE_ID	SALE_DATE	AMOUNT	CUSTOMER_ID	CUSTOMER_NAME
1	408	2/18/2025, 12:00:0(	8852	9	Cust_9
2	409	2/19/2025, 12:00:0(	2366	10	Cust_10
3	410	2/20/2025, 12:00:0(	3858	11	Cust_11
4	411	2/21/2025, 12:00:0(	8599	12	Cust_12
5	412	2/22/2025, 12:00:0(	2221	13	Cust_13

61. Join sales with stores table.

```
SELECT s.sale_id,
       s.amount,
       s.store_id,
       st.store_name
FROM sales5 s
LEFT JOIN stores5 st
ON s.store_id = st.store_id;
```

Output:

	SALE_ID	AMOUNT	STORE_ID	STORE_NAME
1	408	8852	1	CMR
2	409	2366	2	SR
3	410	3858	3	Lucky
4	411	8599	1	CMR
5	412	2221	2	SR

62. Join sales with products table.

```
SELECT s.sale_id,
       s.amount,
       p.product_id,
       p.category
FROM sales5 s
LEFT JOIN products5 p
ON s.product_id = p.product_id;
```

Output:

	SALE_ID	AMOUNT	PRODUCT_ID	CATEGORY
1	408	8852	1	Electronics
2	409	2366	2	Fashion
3	410	3858	3	Grocery
4	411	8599	4	Home
5	412	2221	1	Electronics

63. Find customer name with store name and sales.

```
SELECT c.customer_name,
       st.store_name,
       s.amount
FROM sales5 s
JOIN customers5 c
ON s.customer_id = c.customer_id
JOIN stores5 st
ON s.store_id = st.store_id;
```

Output:

	CUSTOMER_NAME	STORE_NAME	AMOUNT
1	Cust_9	CMR	8852
2	Cust_10	SR	2366
3	Cust_11	Lucky	3858
4	Cust_12	CMR	8599
5	Cust_13	SR	2221

64. Find category-wise revenue using joins.

```
SELECT p.category,
    SUM(s.amount) AS revenue
FROM sales5 s
JOIN products5 p
ON s.product_id = p.product_id
GROUP BY p.category;
```

Output:

	CATEGORY	REVENUE
1	Electronics	1254138
2	Fashion	1296327
3	Grocery	1202791
4	Home	1302264

65. Find store-wise customer count

```
SELECT st.store_id,
    st.store_name,
    COUNT(DISTINCT s.customer_id) AS customer_count
FROM sales5 s
JOIN stores5 st
ON s.store_id = st.store_id
GROUP BY st.store_id, st.store_name
```

Output:

	STORE_ID	STORE_NAME	CUSTOMER_COUNT
1	1	CMR	50
2	2	SR	50
3	3	Lucky	50

66. Find top product per store.

```
SELECT st.store_id,
    st.store_name,
    COUNT(DISTINCT s.customer_id) AS customer_count
FROM sales5 s
JOIN stores5 st
ON s.store_id = st.store_id
GROUP BY st.store_id, st.store_name
```

Output:

	STORE_ID	STORE_NAME	CUSTOMER_COUNT
1	1	CMR	50
2	2	SR	50
3	3	Lucky	50

67. Find customers who purchased from multiple stores.

```
SELECT customer_id,
    COUNT(DISTINCT store_id) AS store_count
FROM sales5
GROUP BY customer_id
HAVING COUNT(DISTINCT store_id) > 1;
```

Output:

Clear output

	STOMER_ID	STORE_COUNT
1	9	3
2	10	3
3	11	3
4	12	3
5	13	3

68. Find sales with product category

```
SELECT s.sale_id,
    s.amount,
    p.category
  FROM sales5 s
  JOIN products5 p
  ON s.product_id = p.product_id
```

Output:

	SALE_ID	AMOUNT	CATEGORY
1	408	8852	Electronics
2	409	2366	Fashion
3	410	3858	Grocery
4	411	8599	Home
5	412	2221	Electronics

69. Find revenue per customer per store.

```
SELECT s.customer_id,
    s.store_id,
    SUM(s.amount) AS revenue
FROM sales5 s
GROUP BY s.customer_id, s.store_id;
```

Output:

	CUSTOMER_ID	STORE_ID	REVENUE
1	9	1	34220
2	10	2	34678
3	11	3	26766
4	12	1	34415
5	13	2	21407

70. Find customers with no transactions (LEFT JOIN).

```
SELECT c.customer_id,
    c.customer_name
FROM customers5 c
LEFT JOIN sales5 s
ON c.customer_id = s.customer_id
WHERE s.customer_id IS NULL;
```

Output:

CUSTOMER_ID	CUSTOMER_NAME

No items to display.

71. Find missing product mappings.

```
SELECT s.sale_id,
    s.product_id
FROM sales5 s
LEFT JOIN products5 p
ON s.product_id = p.product_id
WHERE p.product_id IS NULL
```

Output:

SALE_ID	PRODUCT_ID

No items to display.

72. Find store performance using joins.

```
SELECT st.store_id,
    st.store_name,
    SUM(s.amount) AS revenue,
    COUNT(s.sale_id) AS transactions
FROM stores5 st
LEFT JOIN sales5 s
ON st.store_id = s.store_id
GROUP BY st.store_id, st.store_name;
```

Output:

	STORE_ID	STORE_NAME	REVENUE	TRANSACTIONS
1	1	CMR	1651758	333
2	2	SR	1718445	334
3	3	Lucky	1685317	333

73. Find cross-category purchases.

```
SELECT customer_id,
    COUNT(DISTINCT p.category) AS category_count
FROM sales5 s
JOIN products5 p
ON s.product_id = p.product_id
GROUP BY customer_id
HAVING COUNT(DISTINCT p.category) > 1;
```

Output:

	CUSTOMER_ID	CATEGORY_COUNT
1	9	2
2	10	2
3	11	2
4	12	2
5	13	2

74. Find total revenue using joins.

```
SELECT SUM(s.amount) AS total_revenue
FROM sales5 s
JOIN products5 p
ON s.product_id = p.product_id;
```

Output:

	TOTAL_REVENUE
1	5055520

---

8. Window Functions

Manager Requirement:
Provide advanced analytics without losing row-level data.

Questions:

75. Calculate total spend per customer using window function.

```
SELECT sale_id,
    customer_id,
    amount,
    SUM(amount) OVER (PARTITION BY customer_id) AS total_spent
FROM sales5;
```

Output:

	SALE_ID	CUSTOMER_ID	AMOUNT	TOTAL_SPENT
1	450	1	5343	101258
2	500	1	9128	101258
3	550	1	6998	101258
4	600	1	1623	101258
5	650	1	5257	101258

76. Rank customers by revenue.

```
SELECT customer_id,
    SUM(amount) AS revenue,
     RANK() OVER (ORDER BY SUM(amount) DESC) AS rank
FROM sales5
GROUP BY customer_id;
```

Output:

	CUSTOMER_ID	REVENUE	RANK
1	22	127374	1
2	36	125249	2
3	18	125166	3
4	46	125139	4
5	47	123382	5

77. Find top customer per store.

```
SELECT store_id,
    customer_id,
    total_spent
FROM (
  SELECT store_id,
      customer_id,
      SUM(amount) AS total_spent,
      RANK() OVER (PARTITION BY store_id ORDER BY SUM(amount) DESC) AS rnk
  FROM sales5
  GROUP BY store_id, customer_id
)
WHERE rnk = 1;
```

Output:

	STORE_ID	CUSTOMER_ID	TOTAL_SPENT
1	1	10	47581
2	2	42	50606
3	3	18	47446

78. Calculate running total of sales.

```
SELECT sale_date,
```

```
    amount,
    SUM(amount) OVER (ORDER BY sale_date) AS running_total
FROM sales5;
```

Output:

	SALE_DATE	AMOUNT	RUNNING_TOTAL
1	1/1/2025, 12:00:00	8812	49096
2	1/1/2025, 12:00:00	640	49096
3	1/1/2025, 12:00:00	2835	49096
4	1/1/2025, 12:00:00	7907	49096
5	1/1/2025, 12:00:00	6573	49096

79. Find previous sale using LAG.

```
SELECT sale_id,
    sale_date,
    amount,
    LAG(amount) OVER (ORDER BY sale_date) AS previous_sale
FROM sales5;
```

Output:

	SALE_ID	SALE_DATE	AMOUNT	PREVIOUS_SALE
1	360	1/1/2025, 12:00:00	8812	*(null)*
2	180	1/1/2025, 12:00:00	640	8812
3	270	1/1/2025, 12:00:00	2835	640
4	90	1/1/2025, 12:00:00	7907	2835
5	900	1/1/2025, 12:00:00	6573	7907

80. Find next sale using LEAD.

```
SELECT sale_id,
    sale_date,
    amount,
LEAD(amount) OVER (ORDER BY sale_date) AS next_sale
FROM sales5;
```

Output:

	SALE_ID	SALE_DATE	AMOUNT	NEXT_SALE
1	360	1/1/2025, 12:00:00	8812	640
2	180	1/1/2025, 12:00:00	640	2835
3	270	1/1/2025, 12:00:00	2835	7907
4	90	1/1/2025, 12:00:00	7907	6573
5	900	1/1/2025, 12:00:00	6573	1163

81. Calculate sales growth.

```
SELECT sale_date,
    amount,
    amount - LAG(amount) OVER (ORDER BY sale_date) AS growth
```

```
FROM sales5;
```

Output:

	SALE_DATE	AMOUNT	GROWTH
1	1/1/2025, 12:00:00	8812	(null)
2	1/1/2025, 12:00:00	640	-8172
3	1/1/2025, 12:00:00	2835	2195
4	1/1/2025, 12:00:00	7907	5072
5	1/1/2025, 12:00:00	6573	-1334

82. Find difference between transactions.

```
SELECT sale_id,
    amount,
    amount - LAG(amount) OVER (ORDER BY sale_id) AS difference
FROM sales5;
```

Output:

	SALE_ID	AMOUNT	DIFFERENCE
1	1	2645	(null)
2	2	1358	-1287
3	3	5181	3823
4	4	3642	-1539
5	5	7611	3969

83. Calculate cumulative revenue.

```
SELECT sale_date,
    SUM(amount) OVER (ORDER BY sale_date) AS cumulative_revenue
FROM sales5;
```

Output:

	SALE_DATE	CUMULATIVE_REVE
1	1/1/2025, 12:00:00	49096
2	1/1/2025, 12:00:00	49096
3	1/1/2025, 12:00:00	49096
4	1/1/2025, 12:00:00	49096
5	1/1/2025, 12:00:00	49096

84. Find moving average of sales.

```
SELECT sale_date,
    amount,
AVG(amount) OVER (ORDER BY sale_date ROWS BETWEEN 2 PRECEDING AND
CURRENT ROW) AS moving_avg
FROM sales5;
```

Output:

	SALE_DATE	AMOUNT	MOVING_AVG
1	1/1/2025, 12:00:00	8812	8812
2	1/1/2025, 12:00:00	640	4726
3	1/1/2025, 12:00:00	2835	95.6666666666665
4	1/1/2025, 12:00:00	7907	3794
5	1/1/2025, 12:00:00	6573	771.666666666667

85. Rank stores by revenue.

```
SELECT sale_date,
    amount,
    AVG(amount) OVER (ORDER BY sale_date ROWS BETWEEN 2 PRECEDING AND
CURRENT ROW) AS moving_avg
FROM sales5;
```

Output:

	STORE_ID	REVENUE	RANK
1	2	1718445	1
2	3	1685317	2
3	1	1651758	3

86. Find second highest sale.

```
SELECT *
FROM (
  SELECT sale_id,
      amount,
      RANK() OVER (ORDER BY amount DESC) AS rnk
  FROM sales5
)
WHERE rnk = 2;
```

Output:

	SALE_ID	AMOUNT	RNK
1	840	9967	2

87. Partition data by store and analyze.

```
SELECT store_id,
    sale_id,
    amount,
    SUM(amount) OVER (PARTITION BY store_id) AS store_total
FROM sales5;
```

Output:

	STORE_ID	SALE_ID	AMOUNT	STORE_TOTAL
1	1	369	9288	1651758
2	1	372	3848	1651758
3	1	375	3878	1651758
4	1	378	9545	1651758
5	1	381	5665	1651758

88. Find customer ranking per store.

```
SELECT store_id,
    customer_id,
    SUM(amount) AS revenue,
    RANK() OVER (PARTITION BY store_id ORDER BY SUM(amount) DESC) AS rank
FROM sales5
GROUP BY store_id, customer_id;
```

Output:

	STORE_ID	CUSTOMER_ID	REVENUE	RANK
1	1	10	47581	1
2	1	22	46390	2
3	1	13	44791	3
4	1	17	43787	4
5	1	47	42981	5

89. Calculate frequency using window

```
SELECT sale_id,
    customer_id,
    COUNT(*) OVER (PARTITION BY customer_id) AS frequency
FROM sales5;
```

output:

	SALE_ID	CUSTOMER_ID	FREQUENCY
1	450	1	20
2	500	1	20
3	550	1	20
4	600	1	20
5	650	1	20

---

9. NTILE / Segmentation

Manager Requirement:
Segment customers for marketing.

Questions:

90. Divide customers into 4 segments.

```
SELECT customer_id,
    SUM(amount) AS revenue,
    NTILE(4) OVER (ORDER BY SUM(amount) DESC) AS segment
FROM sales5
GROUP BY customer_id;
```

Output:

	CUSTOMER_ID	REVENUE	SEGMENT
1	22	127374	1
2	36	125249	1
3	18	125166	1
4	46	125139	1
5	47	123382	1

91. Identify top 25% customers.

```
SELECT *
FROM (
  SELECT customer_id,
      SUM(amount) AS revenue,
      NTILE(4) OVER (ORDER BY SUM(amount) DESC) AS seg
  FROM sales5
  GROUP BY customer_id
)
WHERE seg = 1;
```

Output:

	CUSTOMER_ID	REVENUE	SEG
1	22	127374	1
2	36	125249	1
3	18	125166	1
4	46	125139	1
5	47	123382	1

92. Identify bottom 25% customers.

```
SELECT *
FROM (
  SELECT customer_id,
      SUM(amount) AS revenue,
      NTILE(4) OVER (ORDER BY SUM(amount) DESC) AS seg
  FROM sales5
  GROUP BY customer_id
)
WHERE seg = 4;
```

Output:

	CUSTOMER_ID	REVENUE	SEG
1	38	94176	4
2	49	93982	4
3	9	93677	4
4	7	92161	4
5	35	91700	4

93. Segment stores into 3 groups.

```
SELECT store_id,
SUM(amount) AS revenue,
    NTILE(3) OVER (ORDER BY SUM(amount) DESC) AS segment
FROM sales5
GROUP BY store_id;
```

Output:

	STORE_ID	REVENUE	SEGMENT
1	2	1718445	1
2	3	1685317	2
3	1	1651758	3

94. Create quartiles based on revenue.

```
SELECT customer_id,
    SUM(amount) AS revenue,
    NTILE(4) OVER (ORDER BY SUM(amount)) AS quartile
FROM sales5
GROUP BY customer_id;
```

Output:

	CUSTOMER_ID	REVENUE	QUARTILE
1	28	63709	1
2	21	77903	1
3	5	81683	1
4	43	82056	1
5	8	82955	1

95. Segment customers per store.

```
SELECT store_id,
    customer_id,
    SUM(amount) AS revenue,
    NTILE(3) OVER (PARTITION BY store_id ORDER BY SUM(amount) DESC) AS
segment
FROM sales5
GROUP BY store_id, customer_id;
```

Output:

	STORE_ID	CUSTOMER_ID	REVENUE	SEGMENT
1	1	10	47581	1
2	1	22	46390	1
3	1	13	44791	1
4	1	17	43787	1
5	1	47	42981	1

96. Identify premium segment.

```
SELECT *
FROM (
  SELECT customer_id,
      SUM(amount) AS revenue,
      NTILE(4) OVER (ORDER BY SUM(amount) DESC) AS seg
  FROM sales5
  GROUP BY customer_id
)
WHERE seg = 1;
```

Output:

	CUSTOMER_ID	REVENUE	SEG
1	22	127374	1
2	36	125249	1
3	18	125166	1
4	46	125139	1
5	47	123382	1

97. Find mid-level customers.

```
SELECT *
FROM (
  SELECT customer_id,
      SUM(amount) AS revenue,
      NTILE(4) OVER (ORDER BY SUM(amount) DESC) AS seg
  FROM sales5
  GROUP BY customer_id
)
WHERE seg IN (2,3);
```

Output:

	CUSTOMER_ID	REVENUE	SEG
1	29	105889	2
2	17	105483	2
3	20	105362	2
4	39	104615	2
5	13	103449	2

98. Analyze bucket distribution.

```
SELECT segment,
    COUNT(*) AS customer_count
FROM (
  SELECT customer_id,
      NTILE(4) OVER (ORDER BY SUM(amount) DESC) AS segment
  FROM sales5
  GROUP BY customer_id
)
GROUP BY segment;
```

Output:

	SEGMENT	CUSTOMER_COUNT
1	1	13
2	2	13
3	3	12
4	4	12

99. Create marketing segments.

```
SELECT customer_id,
    SUM(amount) AS revenue,
    CASE
      WHEN SUM(amount) > 50000 THEN 'Premium'
      WHEN SUM(amount) > 20000 THEN 'Gold'
      WHEN SUM(amount) > 10000 THEN 'Silver'
      ELSE 'Basic'
    END AS marketing_segment
FROM sales5
GROUP BY customer_id;
```

Output:

	CUSTOMER_ID	REVENUE	MARKETING_SEGM
1	9	93677	Premium
2	10	115763	Premium
3	11	94592	Premium
4	12	106733	Premium
5	13	103449	Premium

---

10. FIRST_VALUE / LAST_VALUE

Manager Requirement:
Compare performance with best and worst.

Questions:

100. Find highest sale.

```
SELECT MAX(amount) AS highest_sale
FROM sales5;
```

Output:

	HIGHEST_SALE
1	9978

101. Find lowest sale.

```
SELECT MIN(amount) AS lowest_sale
FROM sales5
```

Output:

	LOWEST_SALE
1	101

102. Compare each sale with highest.

```
SELECT sale_id,
    amount,
    MAX(amount) OVER () AS highest_sale,
    amount - MAX(amount) OVER () AS difference
FROM sales5;
```

Output:

	SALE_ID	AMOUNT	HIGHEST_SALE	DIFFERENCE
1	408	8852	9978	-1126
2	409	2366	9978	-7612
3	410	3858	9978	-6120
4	411	8599	9978	-1379
5	412	2221	9978	-7757

103. Compare each sale with lowest.

```
SELECT sale_id,
    amount,
    MIN(amount) OVER () AS lowest_sale,
    amount - MIN(amount) OVER () AS difference
FROM sales5;
```

Output:

	SALE_ID	AMOUNT	LOWEST_SALE	DIFFERENCE
1	408	8852	101	8751
2	409	2366	101	2265
3	410	3858	101	3757
4	411	8599	101	8498
5	412	2221	101	2120

104. Find highest sale per store.

```
SELECT store_id,
    MAX(amount) AS highest_sale
FROM sales5
 GROUP BY store_id;
```

Output:

	STORE_ID	HIGHEST_SALE
1	1	9978
2	2	9960
3	3	9966

105. Find lowest sale per store.

```
SELECT store_id,
    MIN(amount) AS lowest_sale
FROM sales5
GROUP BY store_id;
```

Output:

	STORE_ID	LOWEST_SALE
1	1	117
2	2	101
3	3	141

106. Compare category performance.

```
SELECT p.category,
    SUM(s.amount) AS revenue,
    RANK() OVER (ORDER BY SUM(s.amount) DESC) AS rank
FROM sales5 s
JOIN products5 p
ON s.product_id = p.product_id
GROUP BY p.category;
```

Output:

	CATEGORY	REVENUE	RANK
1	Home	1302264	1
2	Fashion	1296327	2
3	Electronics	1254138	3
4	Grocery	1202791	4

107. Find best customer per store.

```
SELECT store_id,
     customer_id,
     total_spent
FROM (
  SELECT store_id,
       customer_id,
       SUM(amount) AS total_spent,
       RANK() OVER (PARTITION BY store_id ORDER BY SUM(amount) DESC) AS rnk
  FROM sales5
  GROUP BY store_id, customer_id
)
WHERE rnk = 1;
```

Output:

	STORE_ID	CUSTOMER_ID	TOTAL_SPENT
1	1	10	47581
2	2	42	50606
3	3	18	47446

108. Find worst performing category.

```
SELECT *
FROM (
  SELECT p.category,
       SUM(s.amount) AS revenue,
       RANK() OVER (ORDER BY SUM(s.amount)) AS rnk
  FROM sales5 s
  JOIN products5 p
  ON s.product_id = p.product_id
  GROUP BY p.category
)
WHERE rnk = 1;
```

Output:

	CATEGORY	REVENUE	RNK
1	Grocery	1202791	1

109. Analyze gap between best and worst.

```
SELECT
  MAX(amount) AS highest_sale,
  MIN(amount) AS lowest_sale,
  MAX(amount) - MIN(amount) AS gap
FROM sales5;
```

Output:

	HIGHEST_SALE	LOWEST_SALE	GAP
1	9978	101	9877

---

11. Date Analysis

Manager Requirement:
Analyze time-based trends.

Questions:

110. Find daily sales.

```
SELECT sale_date,
    SUM(amount) AS daily_sales
FROM sales5
GROUP BY sale_date
ORDER BY sale_date
```

Output:

	SALE_DATE	DAILY_SALES
1	1/1/2025, 12:00:00	49096
2	1/2/2025, 12:00:00	63773
3	1/3/2025, 12:00:00	72872
4	1/4/2025, 12:00:00	55521
5	1/5/2025, 12:00:00	65676

111. Find monthly sales.

```
SELECT TO_CHAR(sale_date,'YYYY-MM') AS month,
 SUM(amount) AS monthly_sales
FROM sales5
GROUP BY TO_CHAR(sale_date,'YYYY-MM')
ORDER BY month;
```

Output:

	MONTH	MONTHLY_SALES
1	2025-01	1813216
2	2025-02	1565707
3	2025-03	1676597

112. Find weekend sales.

```
SELECT sale_id,
    sale_date,
    amount
FROM sales5
WHERE TO_CHAR(sale_date,'DY') IN ('SAT','SUN');
```

Output:

	SALE_ID	SALE_DATE	AMOUNT
1	412	2/22/2025, 12:00:0(	2221
2	413	2/23/2025, 12:00:0(	8529
3	419	3/1/2025, 12:00:00	6901
4	420	3/2/2025, 12:00:00	1036
5	426	3/8/2025, 12:00:00	5408

113. Find first sale date.

```
SELECT MIN(sale_date) AS first_sale_date
FROM sales5;
```

Output:

	FIRST_SALE_DATE
1	1/1/2025, 12:00:00

114. Find last sale date.

```
SELECT MAX(sale_date) AS last_sale_date
FROM sales5;
```

Output:

	LAST_SALE_DATE
1	3/31/2025, 12:00:00

115. Find sales in specific month.

```
SELECT *
FROM sales5
WHERE TO_CHAR(sale_date,'MM-YYYY') = '01-2025';
```

Output:

	SALE_ID	CUSTOMER_ID	STORE_ID	PRODUCT_ID	SALE_DATE	AMOUNT	QUANTITY	DISCOUNT
1	450	1	1	3	1/1/2025, 12:00:00	5343	1	187
2	451	2	2	4	1/2/2025, 12:00:00	6840	2	362
3	452	3	3	1	1/3/2025, 12:00:00	9813	3	(null)
4	453	4	1	2	1/4/2025, 12:00:00	4552	4	77
5	454	5	2	3	1/5/2025, 12:00:00	4064	5	140

116. Find sales growth month-wise.

```
SELECT month,
    monthly_sales,
    LAG(monthly_sales) OVER (ORDER BY month) AS prev_month_sales,
    monthly_sales - LAG(monthly_sales) OVER (ORDER BY month) AS growth
FROM (
  SELECT TO_CHAR(sale_date,'YYYY-MM') AS month,
      SUM(amount) AS monthly_sales
  FROM sales5
  GROUP BY TO_CHAR(sale_date,'YYYY-MM')
);
```

Output:

	MONTH	MONTHLY_SALES	PREV_MONTH_SAL	GROWTH
1	2025-01	1813216	(null)	(null)
2	2025-02	1565707	1813216	-247509
3	2025-03	1676597	1565707	110890

117. Find inactive customers (30 days).

```
SELECT customer_id
FROM sales5
GROUP BY customer_id
HAVING MAX(sale_date) < SYSDATE - 30;
```

Output:

	CUSTOMER_ID
1	9
2	10
3	11
4	12
5	13

118. Find repeat customers.

```
SELECT customer_id,
    COUNT(*) AS transactions
FROM sales5
GROUP BY customer_id
HAVING COUNT(*) > 1;
```

Output:

	CUSTOMER_ID	TRANSACTIONS
1	9	20
2	10	20
3	11	20
4	12	20
5	13	20

119. Find sales trend.

```
SELECT sale_date,
    SUM(amount) AS daily_sales,
    AVG(SUM(amount)) OVER (ORDER BY sale_date ROWS BETWEEN 2 PRECEDING
AND CURRENT ROW) AS moving_avg
FROM sales5
GROUP BY sale_date
ORDER BY sale_date;
```

Output:

	SALE_DATE	DAILY_SALES	MOVING_AVG
1	1/1/2025, 12:00:00	49096	49096
2	1/2/2025, 12:00:00	63773	56434.5
3	1/3/2025, 12:00:00	72872	913.666666666664
4	1/4/2025, 12:00:00	55521	1055.333333333336
5	1/5/2025, 12:00:00	65676	1689.666666666664